Simplified Load Model

Load = Route. Shipment = Truck. That's it.

One insight that eliminates 80% of the edge cases:

All shipments in a load share the same route.
Different routes = different loads.

March 2026 | Architecture Simplification

The Core Insight

Load = Route

A Load defines: which clients to visit (sources) + where to deliver (destination) + the pickup sequence. This is **the route**.

Shipment = Truck on that Route

A Shipment is just a truck assigned to run the Load's route. It has: transporter, vehicle, driver. Nothing else. **All trucks follow the same route.**

Different Route = Different Load

"I want Daikin and Deloitte on one route, Infosys on a different route" = 2 separate loads. Not one load with 2 shipments going different directions.

What This Changes

BEFORE (COMPLEX)

→ Each shipment has its own stops

→ PRs assigned to specific shipments

→ Add PR → "which shipment?"

→ Add Shipment → clone which stops?

→ Milk Run + Multi-Vehicle = confusion

→ Pattern detection = 6+ edge cases

→ Stops owned by shipment

AFTER (SIMPLE)

→ Route defined at Load level

→ PRs belong to the Load (the route)

→ Add PR → adds to route. All trucks get it.

→ Add Shipment → new truck, same route.

→ Patterns auto-detected, no conflicts

→ Pattern = count of sources × trucks

→ Stops owned by Load, executed by shipments

Every Action — Simplified

ACTION	WHAT HAPPENS	PATTERN UPDATE
+ Add PR	New PICKUP stop added to the route. **All trucks** get this stop.	Auto-detect from source count × truck count
+ Add Shipment (Truck)	New truck on the **same route**. Same pickups, same destination.	Auto-detect
- Remove PR	PICKUP stop removed from route. **Removed from all trucks.**	Auto-detect
- Remove Shipment	Truck removed. Route unchanged.	Auto-detect
Change Destination	**All trucks** go to new destination.	Unchanged
Reorder Pickups	Sequence changes for **all trucks**.	Unchanged

Zero ambiguity. Every action has exactly ONE outcome. No "which shipment?" questions. No edge cases.

Pattern Auto-Detection

SOURCES	TRUCKS	PATTERN	EXAMPLE
1 PR	1	**Direct**	One client, one truck
2+ PRs	1	**Milk Run**	3 clients, one truck on a route
1 PR	2+	**Multi-Vehicle**	Big pickup, 2 trucks
2+ PRs	2+	**Milk Run Multi-Vehicle**	3 clients, 2 trucks on same route
WH source	any	**Warehouse Consolidation**	Moving material from warehouse
Cross-dock is a separate creation flow (Slice 5) — not a mutation of standard loads			

Flow Diagram — Before vs After

Before: Each Shipment Has Its Own Stops

```
Complex — 3 PRs, 2 Trucks:

[Daikin]  ──→ 🚚 Ship 1 (PICKUP Daikin + PICKUP Deloitte + DELIVER) ──→ [Plant]
[Deloitte] ──┘
[Infosys] ──→ 🚚 Ship 2 (PICKUP Infosys + DELIVER) ──→ [Plant]

❌ PRs assigned to SPECIFIC shipments
❌ What if user wants to move Infosys to Ship 1?
❌ Stops duplicated per shipment
❌ Add PR → "which ship?" question
```

After: Route Belongs to Load, Trucks Just Run It

```
Simple — 3 PRs, 2 Trucks:

           ┌── ROUTE ──────────────────┐
[Daikin]   │  1. Pickup Daikin         │
[Deloitte] │  2. Pickup Deloitte       │ ──→ [Plant]
[Infosys]  │  3. Pickup Infosys        │
           └───────────────────────────┘
                      │
              ┌───────┴───────┐
              🚚1     🚚2
          (same route, different trucks)
          (material splits at reality)

✅ Route defined ONCE at Load level
✅ All trucks follow same route
✅ Add PR → route updates for all
✅ Add Truck → same route, new truck
✅ No "which ship?" questions ever
```

Data Model Changes

What Moves from Shipment to Load

FIELD	BEFORE (ON SHIPMENT)	AFTER (ON LOAD)
stops[]	Each shipment has own stops	**Load has the route (stops). Shipments reference it.**
pickup sequence	Per-shipment stop order	**One sequence for the Load. All trucks follow it.**
prlds[]	Implicit via stop.prld per shipment	**Explicit on Load (already exists)**

Shipment Becomes Simpler

FIELD	STAYS	WHY
id	✅	Identity
loadId	✅	Which load/route
transporterName	✅	Truck-specific
vehicleRegistration	✅	Truck-specific
driverName + phone	✅	Truck-specific
status	✅	Draft/Planned/In Transit/Completed
~~stopIds[]~~	Moved to Load	Route is shared

Execution nuance: At execution time, each truck records its OWN actual items per stop. So the Stop entity gets a `shipmentActuals` map: `{ [shipmentId]: { actualItems, weights } }`. The planned route is shared, the actuals are per-truck.

What This Eliminates

Killed Edge Cases

#	EDGE CASE	BEFORE	AFTER
1	Milk Run + Add Shipment	Clone which stops? All? Some?	Just add truck. Same route.
2	Multi-Vehicle + Add PR	Which shipment gets the PR?	Route updates. All trucks get it.
3	Move PR between shipments	Complex drag-and-drop UX	Not needed. Shared route.
4	Remove PR from one shipment	What about other shipments?	Removed from route = all trucks.
5	Inconsistent stop counts	Ship 1: 3 stops, Ship 2: 1 stop	Impossible. All trucks = same stops.
6	Pattern hybrid confusion	"Milk Run Multi-Vehicle" = ???	3 sources, 2 trucks. Clear.
7	Stop reorder per shipment	Different order per truck?	One order. All trucks follow it.
8	addShipmentToLoad() complexity	Clone stops, assign PRs, detect pattern	Just create truck entity. Done.

Reduced Code

COMPONENT	BEFORE	AFTER
addShipmentToLoad()	65 lines (clone stops, assign PRs)	~10 lines (create truck)
addPRsToLoad()	Complex (create stops per shipment)	Simple (add to route)
removePRFromLoad()	Delete from all shipments' stops	Remove from route
FlowLayout	Per-shipment stop tracking	Shared route display
ShipmentDetail	Own stops per shipment	Shared route, per-truck actuals
Pattern detection	Complex rules with edge cases	# sources × # trucks matrix

Execution Model — How Actuals Work

Planning: Route is shared. All trucks see the same pickup sequence.

Execution: Each truck records its OWN actual quantities at each stop. Because in reality, Truck 1 might pick 6T at Daikin while Truck 2 picks 4T.

Example: Milk Run + 2 Trucks

Route (planned):

```
1. PICKUP Daikin (Ahmedabad) — planned: 6,200 kg
2. PICKUP Deloitte (Hyderabad) — planned: 4,000 kg
3. PICKUP Infosys (Pune) — planned: 6,000 kg
4. DELIVER Plant (Haridwar)
```

Truck 1 actuals (at execution):

STOP	PLANNED	TRUCK 1 ACTUAL
Daikin	6,200 kg	**4,000 kg**
Deloitte	4,000 kg	**2,500 kg**
Infosys	6,000 kg	**3,500 kg**
Plant (deliver)	—	Net: 10,000 kg

Truck 2 actuals:

STOP	PLANNED	TRUCK 2 ACTUAL
Daikin	6,200 kg	**2,200 kg**
Deloitte	4,000 kg	**1,500 kg**
Infosys	6,000 kg	**2,500 kg**
Plant (deliver)	—	Net: 6,200 kg

PR closure check:

PR	PLANNED	TOTAL PICKED (T1+T2)	STATUS
Daikin	6,200 kg	4,000 + 2,200 = 6,200 kg	✅ Closed
Deloitte	4,000 kg	2,500 + 1,500 = 4,000 kg	✅ Closed

Infosys	6,000 kg	3,500 + 2,500 = 6,000 kg	✅ Closed

Cross-Dock — Separate by Design

> **Cross-dock is NOT "same route, multiple trucks."** It's "different routes that connect at a hub." This is fundamentally different and should be a separate creation flow.

Standard Load vs Cross-Dock Load

	STANDARD LOAD	CROSS-DOCK LOAD
Route	One shared route	Multiple connected routes
Trucks	All follow same route	Each has different route
Stops	Shared	Per-shipment (feeders vs line-haul)
Hub	No hub	Transfer point connects routes
Creation	"Create Load" (standard)	"Create Cross-Dock" (explicit)
User intent	"Send trucks to pick up stuff"	"Coordinate feeder trucks to meet a big truck"

Cross-dock keeps per-shipment stops because each feeder has a DIFFERENT route. This is correct — it's a multi-route operation by nature.